Lab 3: Architecture Selection Justification

Name: Shakthi Raja G P

SRN: PES1UG25CS843

Problem Statement: #60, Podcast Guest Scheduling and Outline Builder

Architecture Selection

We chose Layered Architecture for the Podcast Guest Scheduling and Outline Builder system. The system is divided into three layers. The Presentation Layer contains the guest booking page and host dashboard. The Application Layer handles booking, availability, outlines, and run-of-show generation. The Data and Integration Layer stores the data and connects to calendar and email services.

Reasons for Choosing Layered Architecture

- Clear separation of work. Each layer has a specific job. Changes to the user interface will not directly affect booking rules, PDF generation, or database operations. This also makes the system easier to test and maintain.
- Suitable for the size of the project. The system has a limited number of features and external services. A layered design is easier to build and deploy than microservices. The calendar and notification adapters can still be changed without rewriting the complete system.

Security Advantage

The Application Layer can check authentication and permissions before allowing an action. For example, only the host should be able to approve an outline or generate a production sheet. The user interface does not connect directly to the database. Calendar and email credentials are also kept inside their adapter components, so they are not exposed to users.

Performance Benefit

The main services run inside the same application, so they do not need several network calls to complete one task. Availability data can be cached, and database queries can be indexed. PDF generation is handled by the Outline and Run-of-Show component, which helps the system meet the one-second target given in NFR-001.

Styles Considered

Client-Server	Only describes the browser-to-server split. It gives no internal structure for slot validation, outline compilation or invite dispatch, so it does not tell me how to divide the system into modules.
Microservices	Separate booking, outline and notification services each owning a store would need distributed transactions to stop double-booking, plus extra deployment and monitoring effort - too heavy for a single-host tool.
Layered (chosen)	Matches the sequential request–response workflow and keeps slot validation and booking inside one ACID transaction.

Layered Architecture gives this system a simple structure while keeping the components secure, organized, and easy to maintain.